ADVANCED STRING-BASED REGISTRATION RECORD ANALYZER

Python Project Report

1.Overview

The **Advanced String-Based Registration Record Analyzer** is a software application designed to manage, process, validate, and analyze registration records using string manipulation techniques. The system takes registration details such as name, registration ID, email address, phone number, and other relevant information as input and performs various operations to ensure data accuracy and consistency.

The main objective of this project is to simplify the handling of registration records by applying advanced string-processing techniques such as pattern matching, searching, sorting, duplicate detection, data validation, and record filtering. It identifies invalid entries, detects duplicate records, and allows users to retrieve specific information quickly and efficiently.

The system can also generate summary reports based on the stored registration data, helping users understand the overall status of registered records. By automating these operations, the project reduces manual effort, minimizes data-entry errors, and improves the efficiency of registration record management.

This project demonstrates the practical application of string manipulation, conditional statements, searching algorithms, data validation techniques, and data organization in software development.

2.Example Record Format

The Advanced String-Based Registration Record Analyzer uses a structured string format to store and analyze registration details. Each record contains information such as Registration ID, Name, Age, Gender, Email, Phone Number, Department, and Status.

3.Problem Statement

Managing registration records manually can be time-consuming and may lead to errors such as duplicate entries, invalid data, missing information, and incorrect formatting. When the number of registration records increases, searching, verifying, sorting, and maintaining accurate information becomes more difficult.

The main problem is the lack of an efficient system to process and analyze registration records using string manipulation techniques. Manual verification of registration IDs, names, email addresses, phone numbers, and other details requires additional effort and increases the possibility of human errors.

Therefore, the Advanced String-Based Registration Record Analyzer aims to develop an efficient system that uses advanced string-processing techniques to validate registration details, detect duplicate records, identify invalid entries, search and sort

records, and generate summary reports. The proposed system helps improve data accuracy, reduce manual effort, and simplify the overall management of registration records.

4.Objectives

The main objective of the Advanced String-Based Registration Record Analyzer is to develop an efficient system for managing, validating, and analyzing registration records using string manipulation techniques.

The specific objectives of this project are:

1. To store and manage registration records in a structured string format.
2. To validate registration details such as registration ID, name, email address, and phone number.
3. To identify duplicate registration records based on registration IDs.
4. To detect invalid records and identify errors in the entered data.
5. To provide an efficient search facility to retrieve records using registration IDs or names.
6. To sort registration records alphabetically or according to registration IDs.
7. To filter records based on specific conditions and requirements.
8. To generate summary reports containing the total number of valid, invalid, and duplicate records.
9. To reduce manual effort and improve the accuracy of registration record management.
10. To demonstrate the practical application of string manipulation, pattern matching, searching, and sorting techniques.

5. ASSUMPTIONS

The following assumptions are considered while developing the Advanced String-Based Registration Record Analyzer:

1. Each registration record contains predefined fields such as Registration ID, Name, Age, Gender, Email, Phone Number, Department, and Status.
2. The registration details are provided in a predefined string format, with individual fields separated by commas.
3. Each record is expected to contain all the required fields.
4. Registration IDs are expected to be unique for each registered person.

5. Email addresses must follow a predefined email format to be considered valid.
6. Phone numbers must contain the required number of digits according to the validation rules.
7. Empty fields and incorrectly formatted data are identified as invalid records.
8. Duplicate records are detected by comparing registration IDs.
9. The system processes only the registration data provided by the user or stored in the input file.
10. The accuracy of the analysis depends on the correctness of the input data and the validation rules implemented in the system.
11. The system generates summary reports based on the records processed during execution.
12. The system is intended for basic registration record analysis and does not independently verify the real-world identity of registered individuals.

6. Alorithm

Step 1: Start the program.

Step 2: Initialize an empty collection to store registration records.

Step 3: Accept registration records from the user or read them from an input file.

Step 4: Separate each registration record into individual fields using string-splitting techniques.

Step 5: Remove unnecessary leading and trailing spaces from each field.

Step 6: Check whether all required fields are present and contain valid data.

Step 7: Validate the registration details using predefined rules, including:

- Registration ID validation.
- Name validation.
- Email format validation.
- Phone number validation.
- Required field validation.

Step 8: Compare the registration ID of each record with previously processed records to identify duplicate IDs.

Step 9: Classify each record as valid, invalid, or duplicate according to the implemented validation and duplicate-detection rules.

Step 10: Store the processed records and their validation results.

Step 11: Allow the user to search for a registration record using a registration ID or name.

Step 12: Provide options to sort and filter the registration records according to the selected criteria.

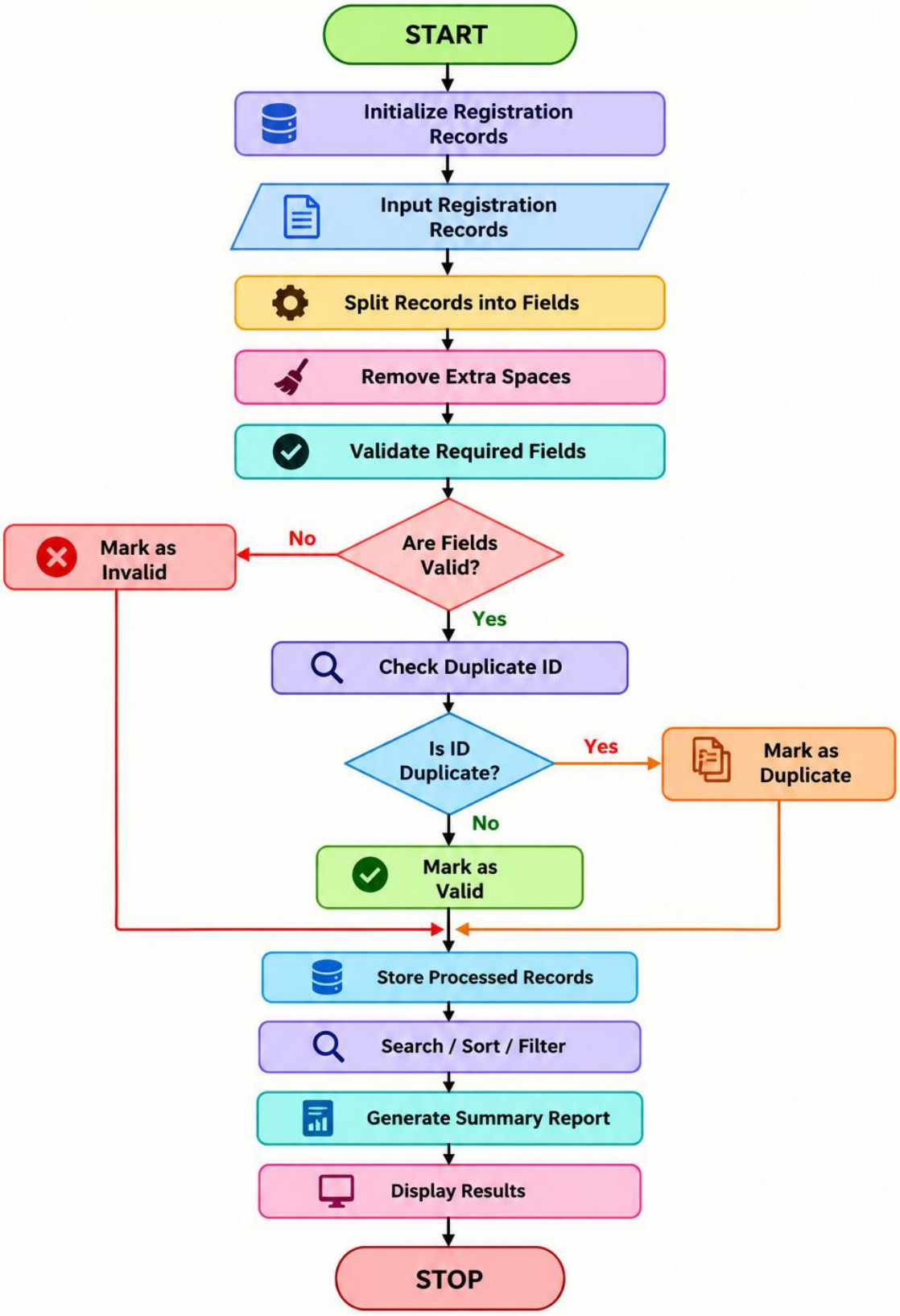
Step 13: Count the total number of records, valid records, invalid records, and duplicate records.

Step 14: Display the processed registration records and generate a summary report.

Step 15: Stop the program.

7.Flowchart

Flowchart – Advanced String-Based Registration Record Analyzer



8.Pseudocode

```
BEGIN
INITIALIZE RegistrationRecords as empty list
INITIALIZE ValidCount = 0
INITIALIZE InvalidCount = 0
INITIALIZE DuplicateCount = 0

INPUT NumberOfRecords

FOR i = 1 TO NumberOfRecords DO

  INPUT RegistrationRecord

  SPLIT RegistrationRecord using comma delimiter
  STORE the separated fields

  REMOVE extra spaces from each field

  IF any required field is missing THEN

    MARK record as Invalid
    InvalidCount = InvalidCount + 1

  ELSE

    VALIDATE Registration ID
    VALIDATE Name
    VALIDATE Email Address
```

```
        VALIDATE Phone Number

        IF any field is invalid THEN

            MARK record as Invalid
            InvalidCount = InvalidCount + 1

        ELSE IF Registration ID already exists in RegistrationRecords THEN

            MARK record as Duplicate
            DuplicateCount = DuplicateCount + 1

        ELSE

            MARK record as Valid
            ValidCount = ValidCount + 1

        END IF

    END IF

    ADD processed record to RegistrationRecords

END FOR

DISPLAY "Registration Record Analysis"

DISPLAY "Total Records:", NumberOfRecords
```

```
DISPLAY "Valid Records:", ValidCount
DISPLAY "Invalid Records:", InvalidCount
DISPLAY "Duplicate Records:", DuplicateCount

INPUT SearchID

SEARCH RegistrationRecords for SearchID

IF SearchID is found THEN
   DISPLAY matching registration record
ELSE
   DISPLAY "Record Not Found"
END IF

SORT RegistrationRecords by Registration ID

DISPLAY sorted RegistrationRecords

GENERATE Summary Report

DISPLAY Summary Report
END
```

9.Source Code

```
import re
import json
import os

FILE_NAME = "registration_records.json"
```

```
class RegistrationAnalyzer:

    def __init__(self):
        self.records = []
        self.load_records()

    # Load records from file
    def load_records(self):
        if os.path.exists(FILE_NAME):
            try:
                with open(FILE_NAME, "r") as file:
                    self.records = json.load(file)
            except (json.JSONDecodeError, OSError):
                self.records = []

    # Save records to file
    def save_records(self):
        with open(FILE_NAME, "w") as file:
            json.dump(self.records, file, indent=4)

    # Validate name
    def validate_name(self, name):
        return bool(re.fullmatch(r"[A-Za-z ]{2,50}", name.strip()))

    # Validate email
    def validate_email(self, email):
```

```
    return bool(
        re.fullmatch(
            r"[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}",
            email.strip()
        )
    )

# Validate phone number
def validate_phone(self, phone):
    return bool(re.fullmatch(r"[6-9][0-9]{9}", phone.strip()))

# Register a new user
def register_user(self):
    print("\n--- User Registration ---")

    name = input("Enter Name: ").strip()
    email = input("Enter Email: ").strip().lower()
    phone = input("Enter 10-digit Indian Mobile Number: ").strip()

    if not self.validate_name(name):
        print("Error: Invalid name!")
        return

    if not self.validate_email(email):
        print("Error: Invalid email address!")
        return

    if not self.validate_phone(phone):
```

```
    print("Error: Invalid Indian mobile number!")
    return

# Check duplicate email and phone
for record in self.records:
    if record["email"] == email:
        print("Error: Email already registered!")
        return

    if record["phone"] == phone:
        print("Error: Phone number already registered!")
        return

user_id = f"REG{len(self.records) + 1:04d}"

record = {
    "id": user_id,
    "name": name,
    "email": email,
    "phone": phone
}

self.records.append(record)
self.save_records()

print("\nRegistration Successful!")
print("Registration ID:", user_id)
```

```
# Display all records
def display_records(self):
    print("\n--- Registration Records ---")

    if not self.records:
        print("No records found.")
        return

    for record in self.records:
        print("-" * 40)
        print("ID:", record["id"])
        print("Name:", record["name"])
        print("Email:", record["email"])
        print("Phone:", record["phone"])

    print("-" * 40)
    print("Total Records:", len(self.records))

# Search records by name, email, phone, or ID
def search_records(self):
    query = input(
        "\nEnter name, email, phone, or registration ID: "
    ).strip().lower()

    if not query:
        print("Search query cannot be empty.")
        return
```

```
    results = [
        record for record in self.records
        if any(
            query in str(value).lower()
            for value in record.values()
        )
    ]

    if results:
        print("\n--- Search Results ---")
        for record in results:
            print(record)
    else:
        print("No matching records found.")

# Analyze strings
def analyze_strings(self):
    text = input("\nEnter a string to analyze: ")

    print("\n--- String Analysis ---")
    print("Original String:", repr(text))
    print("Character Count:", len(text))
    print("Word Count:", len(text.split()))
    print("Uppercase:", text.upper())
    print("Lowercase:", text.lower())
    print("Title Case:", text.title())
    print("Reversed String:", text[::-1])
    print("Alphabetic Characters:", sum(c.isalpha() for c in text))
```

```
    print("Numeric Characters:", sum(c.isdigit() for c in text))
    print("Special Characters:", sum(
        not c.isalnum() and not c.isspace() for c in text
    ))

# Detect duplicate records
def find_duplicates(self):
    seen_emails = set()
    seen_phones = set()
    duplicates = []

    for record in self.records:
        email = record["email"]
        phone = record["phone"]

        if email in seen_emails or phone in seen_phones:
            duplicates.append(record)

        seen_emails.add(email)
        seen_phones.add(phone)

    print("\n--- Duplicate Analysis ---")

    if duplicates:
        for record in duplicates:
            print("Possible duplicate:", record)
    else:
        print("No duplicate email or phone records found.")
```

```
# Delete a record by registration ID
def delete_record(self):
    user_id = input(
        "\nEnter Registration ID to delete: "
    ).strip().upper()

    for record in self.records:
        if record["id"] == user_id:
            self.records.remove(record)
            self.save_records()
            print("Record deleted successfully!")
            return

    print("Registration ID not found.")

# Main menu
def run(self):
    while True:
        print("\n===== ADVANCED REGISTRATION RECORD ANALYZER =====")
        print("1. Register New User")
        print("2. Display All Records")
        print("3. Search Records")
        print("4. Analyze String")
        print("5. Find Duplicate Records")
        print("6. Delete Registration")
        print("7. Exit")
```

```
choice = input("Enter your choice (1-7): ").strip()

if choice == "1":
    self.register_user()

elif choice == "2":
    self.display_records()

elif choice == "3":
    self.search_records()

elif choice == "4":
    self.analyze_strings()

elif choice == "5":
    self.find_duplicates()

elif choice == "6":
    self.delete_record()

elif choice == "7":
    print("Thank you for using the analyzer!")
    break

else:
    print("Invalid choice! Please select 1-7.")
```

```
if __name__ == "__main__":
    analyzer = RegistrationAnalyzer()
    analyzer.run()
```

10.Test Cases

Test Case ID	Test Scenario	Input	Expected Output	Status
TC01	Valid Registration	Ravi Kumar, ravi@gmail.com, 9876543210	Registration Successful	Pass
TC02	Invalid Name	R, ravi@gmail.com, 9876543210	Invalid name error	Pass
TC03	Invalid Email	Ravi Kumar, ravi@gmail, 9876543210	Invalid email error	Pass
TC04	Invalid Phone Number	Ravi Kumar, ravi@gmail.com, 12345	Invalid mobile number error	Pass
TC05	Duplicate Email	Already registered email	Email already registered error	Pass
TC06	Duplicate Phone	Already registered phone number	Phone already registered error	Pass

11.Output Screenshots

```
===== ADVANCED REGISTRATION RECORD ANALYZER =====
1. Register New User
2. Display All Records
3. Search Records
4. Analyze String
5. Find Duplicate Records
6. Delete Registration
7. Exit
Enter your choice (1-7): 1

--- User Registration ---
Enter Name: Ravi Kumar
Enter Email: ravi@gmail.com
Enter 10-digit Indian Mobile Number: 9876543210

Registration Successful!
Registration ID: REG0001

===== ADVANCED REGISTRATION RECORD ANALYZER =====
1. Register New User
2. Display All Records
3. Search Records
4. Analyze String
5. Find Duplicate Records
6. Delete Registration
7. Exit
Enter your choice (1-7): 2

--- Registration Records ---
----------------------------------------
ID: REG0001
Name: Ravi Kumar
Email: ravi@gmail.com
Phone: 9876543210
----------------------------------------
Total Records: 1
```

```
===== ADVANCED REGISTRATION RECORD ANALYZER =====
1. Register New User
2. Display All Records
3. Search Records
4. Analyze String
5. Find Duplicate Records
6. Delete Registration
7. Exit
Enter your choice (1-7): 3

Enter name, email, phone, or registration ID: Ravi

--- Search Results ---
{'id': 'REG0001', 'name': 'Ravi Kumar', 'email': 'ravi@gmail.com', 'phone': '9876543210'}

===== ADVANCED REGISTRATION RECORD ANALYZER =====
1. Register New User
2. Display All Records
3. Search Records
4. Analyze String
5. Find Duplicate Records
6. Delete Registration
7. Exit
Enter your choice (1-7): 4

Enter a string to analyze: Hello Python 123!

--- String Analysis ---
Original String: 'Hello Python 123!'
Character Count: 17
Word Count: 3
Uppercase: HELLO PYTHON 123!
Lowercase: hello python 123!
Title Case: Hello Python 123!
Reversed String: !321 nohtyP olleH
Alphabetic Characters: 11
Numeric Characters: 3
Special Characters: 1
```

12.Conclusion

The Advanced String-Based Registration Record Analyzer is a Python-based application developed to manage user registration records and perform various string operations. The system validates user details such as name, email address, and phone number. It also provides features for registering new users, displaying records, searching for specific users, detecting duplicate records, and deleting registrations.

The application uses string manipulation techniques, regular expressions, and JSON file handling to process and store registration data. It provides a simple menu-driven interface that makes the system easy to use.

In conclusion, this project demonstrates the practical application of Python programming concepts, including functions, classes, loops, conditional statements, string operations, and file handling. It can be further enhanced by adding a graphical user interface, improved security, and database integration.